PropOS Flutter 开发指南

> 文档版本：v1.0
>
> 日期：2026-04-20
>
> 适用范围：flutter*app*/ 目录（*Flutter 3.x + Dart 3 + flutter*bloc + go*router + dio + get*it）
>
> 目标读者：参与 PropOS 移动端开发的 Flutter 开发者

1. 文档定位

本指南以仓库 `flutter_app/` 实际代码为准，说明 PropOS Flutter 移动端的开发方式、目录职责、启动命令和新增功能流程。

相关文档层级：

文档	定位
`.github/instructions/flutter.instructions.md`	Copilot 代码生成约束（机器读）
`docs/frontend/FLUTTER_EXPERT_GUIDE.md`	架构与规范全景（深度参考）
本文档	日常开发操作手册（快速上手）

前提：

1. 仓库移动端实现在 `flutter_app/` （独立 Flutter 工程）。
2. `app/` 为历史 uni-app 项目（已弃用），不再投入开发。
3. `admin/` 为 PC 管理后台（Vue 3 + Element Plus），独立维护。

---

2. 当前项目现状

2.1 已落地部分

- 基础框架：Flutter 3.x、Dart 3（sealed class / pattern matching）
- 状态管理：flutter_bloc ^8.x（BLoC / Cubit + freezed 四态 State）
- 路由：go_router ^14.x
- HTTP：dio ^5.x，统一封装在 `core/api/api_client.dart`
- 依赖注入：get_it ^7.x
- 不可变数据类：freezed + json_serializable
- 日期展示：intl 包 `DateFormat`
- 环境配置：flutter_dotenv（ `.env` / `.env.dev` ）
- 主题：Material 3 + `ThemeExtension<CustomColors>` 语义扩展色

2.2 目标平台

平台	状态	说明
iOS	Phase 1 交付	主力移动端
Android	Phase 1 交付	主力移动端
HarmonyOS Next	Phase 1 交付	鸿蒙原生

> Flutter Web 不在当前交付范围内。

2.3 当前页面状态

go_router 路由表已注册以下页面：

- 认证：登录页
- 总览：Dashboard
- 资产：资产总览、楼栋详情、楼层热区图、房源详情
- 合同：合同列表、合同详情
- 财务：财务总览、账单列表、KPI
- 工单：工单列表、工单详情、新建工单
- 二房东：列表、详情
- 通知中心
- 审批队列

大部分页面为骨架占位，需按模块逐步补齐 Repository、BLoC 和 Widget。

---

3. 技术栈与关键约束

3.1 核心依赖

包	用途	版本
`flutter_bloc`	状态管理	^8.x
`freezed` / `freezed_annotation`	不可变数据类 + sealed union	^2.x
`json_serializable` / `json_annotation`	JSON 序列化	^6.x
`go_router`	声明式路由	^14.x
`dio`	HTTP 客户端	^5.x
`get_it`	依赖注入	^7.x
`intl`	日期/数字格式化	^0.19.x
`flutter_dotenv`	环境变量	^5.x
`bloc_test` + `mocktail`	测试	^9.x / ^1.x

3.2 必须遵守的架构规则

Flutter 端严格遵循 Clean Architecture 单向数据流：

```
ApiClient → Repository(实现) → BLoC/Cubit → Page/Widget
```

禁止出现以下情况：

- 在 Page/Widget 中直接调用 `Dio()` 或 `ApiClient`
- 在 Page/Widget 中硬编码 `/api/contracts` 等接口路径
- 在 BLoC/Cubit 中 `import 'package:flutter/...'`
- 在 presentation 层 `import '../data/...'`
- 在业务代码中直接写 `20` 、 `30` 、 `90` 等魔法数字
- 使用 `if (state is _Loading)` 代替 `switch` / `.when()`
- 直接 `DateTime.now()` （测试不可控），应注入 `Clock`

3.3 编码规范要点

规则	说明
API 路径	`core/api/api_paths.dart` 常量类
业务规则常量	`core/constants/business_rules.dart`
UI 常量	`core/constants/ui_constants.dart` （ `defaultPageSize = 20` ）
路由路径	`core/router/route_paths.dart` 常量类
日期展示	`DateFormat('yyyy-MM-dd').format(dt.toLocal())`
金额	后端传分（int），展示时 `/100`
颜色	`Theme.of(context).colorScheme.*` 或 `extension<CustomColors>()`
错误处理	`e is ApiException ? e.message : '操作失败，请重试'`
State	必须 `@freezed` 四态：initial / loading / loaded / error

---

4. 目录职责

```
flutter_app/lib/
  core/
    api/
      api_client.dart        # dio 封装，含信封解析、JWT 注入、401 刷新
      api_exception.dart     # ApiException(code, message, statusCode)
      api_paths.dart         # 所有 API 端点路径常量
      mock/                  # Mock 拦截器（FLUTTER_USE_MOCK=true 启用）
    constants/
      business_rules.dart    # 业务阈值（预警天数、逾期节点等）
      ui_constants.dart      # 分页大小、动画时长等
    theme/
      app_theme.dart         # Material 3 ThemeData 配置
      custom_colors.dart     # ThemeExtension 语义扩展色
    router/
      app_router.dart        # go_router 路由表 + 守卫
      route_paths.dart       # 路由路径常量
    di/
      injection.dart         # get_it 注册表
    utils/                   # 日期、金额等格式化工具
  features/
    <module>/
      domain/
        entities/            # 纯 Dart 实体（无 Flutter 依赖）
        repositories/        # abstract class（接口契约）
        usecases/            # 单一职责用例（可选）
```

```
    data/
      models/              # @freezed DTO + fromJson + toEntity()
      repositories/        # Repository 实现（调 ApiClient）
    presentation/
      bloc/                # BLoC/Cubit + freezed State
      pages/               # Page Widget（≤ 150 行）
      widgets/             # 子 Widget（≤ 100 行）
  shared/
    widgets/               # 全局共享 Widget（StatusTag、ErrorBlock 等）
    utils/                 # 工具函数
  main.dart                # 入口（DI + runApp）
```

---

5. 本地开发与启动方式

5.1 前置条件

- Flutter SDK 3.x（stable channel）
- Xcode（iOS 开发）
- Android Studio 或 Android SDK（Android 开发）
- 后端接口可访问（默认 `http://localhost:8080`）

```
# 验证环境
flutter doctor -v
```

5.2 安装依赖

```
cd flutter_app

# 安装 Dart/Flutter 依赖
flutter pub get

# 运行代码生成（freezed / json_serializable）
dart run build_runner build --delete-conflicting-outputs
```

5.3 常用启动命令

```
# 列出可用设备
flutter devices

# iOS 模拟器
flutter run -d ios

# Android 模拟器
flutter run -d android

# 指定设备
```

```
flutter run -d <device_id>

# Release 模式（性能测试）
flutter run --release
```

5.4 环境变量

在 `flutter_app/` 根目录创建 `.env` 文件：

```
API_BASE_URL=http://localhost:8080
FLUTTER_USE_MOCK=true
```

- `FLUTTER_USE_MOCK=true` ：启用 Mock 拦截器，无需后端即可开发
- `FLUTTER_USE_MOCK=false` ：连接真实后端 API

5.5 代码生成（持续监听模式）

开发期间修改 `@freezed` 类后需重新生成代码：

```
# 一次性生成
dart run build_runner build --delete-conflicting-outputs

# 持续监听（推荐开发时使用）
dart run build_runner watch --delete-conflicting-outputs
```

---

6. 新增功能模块 SOP

以新增"合同模块"为例，按以下顺序创建文件：

Step 1：Domain 层（纯 Dart，无 Flutter 依赖）

```
features/contracts/domain/
  entities/contract.dart              # 实体类（Dart enum + 纯字段）
  repositories/contract_repository.dart  # abstract class
```

Step 2：Data 层（依赖 ApiClient）

```
features/contracts/data/
  models/contract_model.dart          # @freezed DTO + fromJson + toEntity()
  repositories/contract_repository_impl.dart  # 实现 domain 接口
```

Step 3：Presentation 层（依赖 domain 层）

```
features/contracts/presentation/
  bloc/contract_list_state.dart       # @freezed 四态 State
  bloc/contract_list_cubit.dart       # Cubit（注入 abstract Repository）
  pages/contract_list_page.dart       # BlocProvider + BlocBuilder + switch
```

```
  widgets/contract_card.dart          # 列表项卡片
```

Step 4：注册集成

文件	操作
`core/api/api_paths.dart`	添加 `static const contracts = '/api/contracts'`
`core/di/injection.dart`	注册 `ContractRepositoryImpl` + `ContractListCubit`
`core/router/app_router.dart`	添加 `/contracts` 路由
`core/router/route_paths.dart`	添加 `static const contracts = '/contracts'`

Step 5：Mock + 测试

文件	操作
`core/api/mock/modules/contracts_mock.dart`	Mock 数据（返回信封格式）
`test/features/contracts/bloc/contract_list_cubit_test.dart`	`blocTest` 单元测试

---

7. 常用开发场景速查

7.1 添加新 API 端点

```
// 1. 添加路径常量
// core/api/api_paths.dart
abstract final class ApiPaths {
  static const contracts = '/api/contracts';
  static const contractDetail = '/api/contracts'; // 使用 '${ApiPaths.contracts}/$id'
}

// 2. 在 Repository 实现中调用
Future<ContractDetail> getContractDetail(String id) async {
  return _apiClient.apiGet<ContractDetail>(
    '${ApiPaths.contracts}/$id',
    fromJson: (json) => ContractDetailModel.fromJson(json as Map<String, dynamic>).toEntity(),
  );
}
```

7.2 创建新 freezed State

```
// 1. 创建 state 文件
// features/xxx/presentation/bloc/xxx_state.dart
```

```
import 'package:freezed_annotation/freezed_annotation.dart';

part 'xxx_state.freezed.dart';

@freezed
sealed class XxxState with _$XxxState {
  const factory XxxState.initial() = _Initial;
  const factory XxxState.loading() = _Loading;
  const factory XxxState.loaded(List<Xxx> items, {PaginationMeta? meta}) = _Loaded;
  const factory XxxState.error(String message) = _Error;
}

// 2. 运行代码生成
// dart run build_runner build --delete-conflicting-outputs
```

7.3 添加新页面路由

```
// 1. 添加路径常量
// core/router/route_paths.dart
static const newFeature = '/new-feature';

// 2. 添加路由
// core/router/app_router.dart
GoRoute(
  path: RoutePaths.newFeature,
  builder: (_, __) => const NewFeaturePage(),
),
```

7.4 注册依赖

```
// core/di/injection.dart

// Repository 单例
getIt.registerLazySingleton<NewFeatureRepository>(
  () => NewFeatureRepositoryImpl(getIt<ApiClient>()),
);

// Cubit 工厂（每次创建新实例）
getIt.registerFactory<NewFeatureCubit>(
  () => NewFeatureCubit(getIt<NewFeatureRepository>()),
);
```

7.5 编写 BLoC 测试

```
import 'package:bloc_test/bloc_test.dart';
import 'package:mocktail/mocktail.dart';

class MockRepo extends Mock implements NewFeatureRepository {}

void main() {
```

```
  late MockRepo repo;

  setUp(() => repo = MockRepo());

  blocTest<NewFeatureCubit, NewFeatureState>(
    '成功加载数据',
    build: () {
      when(() => repo.getItems()).thenAnswer((_) async => [testItem]);
      return NewFeatureCubit(repo);
    },
    act: (cubit) => cubit.fetchItems(),
    expect: () => [
      const NewFeatureState.loading(),
      isA<NewFeatureState>(),
    ],
  );
}
```

---

8. 与 Admin 端的关系

维度	Flutter App (`flutter_app/`)	Admin PC (`admin/`)
技术栈	Dart + Flutter + Material 3	TypeScript + Vue 3 + Element Plus
职责	移动端：查看 + 现场轻操作	PC 端：完整工作台 + 批量操作
状态管理	flutter_bloc (BLoC/Cubit)	Pinia (setup)
HTTP	dio → ApiClient	axios → api client
路由	go_router	Vue Router 4
共享	不共享代码，共享后端 API + 设计规范语义色	同左

---

9. 排错速查

问题	解决
`build_runner` 报冲突	`dart run build_runner clean && dart run build_runner build --delete-conflicting-outputs`
`get_it` 找不到实例	检查 `configureDependencies()` 调用顺序 + 注册链完整性
BlocBuilder 不刷新	确认 State 用 freezed（值相等）+ emit 的是新实例
go_router 路由不匹配	子路由 path 不加前导 `/`；检查 redirect 函数
Mock 不生效	确认 `.env` 中 `FLUTTER_USE_MOCK=true` + MockInterceptor 已加载
iOS 模拟器启动慢	`flutter clean && flutter pub get && flutter run`
HarmonyOS 编译失败	确认 SDK 版本 + DevEco 环境配置

10. CI/CD 概要

```
# CI 流水线关键步骤
flutter pub get
flutter analyze                        # 静态分析
flutter test                           # 单元测试
flutter build apk --release            # Android 产物
flutter build ipa --release            # iOS 产物（需 macOS Runner）
```

测试覆盖率：

```
flutter test --coverage
# 产出 coverage/lcov.info
```

详见 `docs/guide/CICD_PIPELINE.md` 。